Online Quiz and Exam System

Complete project documentation: from the specification to a tested application

React JS | FastAPI | PostgreSQL

Built with the Specification Driven Development flow: spec, plan, implement, test, review

22	9	18	30
build steps	React pages	API endpoints	automated tests passed

This document explains how the project was done from start to end: how the topic was chosen, how the specification (SDD) was written, how every part was implemented step by step with screenshots, where I got stuck, how each problem was solved, and how the application was tested and reviewed.

Report date: 30 October 2026

Contents

Contents 2

1. Introduction and approach 4

1.1 The assignment . . . 4

1.2 The development flow . . . 4

1.3 How I worked . . . 4

1.4 The reference document . . . 5

2. Choosing the project 6

2.1 Decision: Online Quiz and Exam System . . . 6

2.2 Scope decision . . . 6

3. The specification (SDD) 7

3.1 Structure compared to the reference . . . 7

3.2 Key rules written in the SDD . . . 7

3.3 Requirement groups . . . 7

4. Tools and environment 8

4.1 Architecture . . . 8

5. Build log, Part A: the React frontend 9

Step 1. Check Node.js . . . 9

Step 2. Create the React app with Vite . . . 9

Step 3. Extra packages and folders . . . 10

Step 4. The API client (src/api/client.js) . . . 10

Step 5. AuthContext (who is logged in) . . . 11

Step 6. Router, protected routes and the home redirect . . . 11

Step 7. The Login page . . . 11

Step 8. Layout with the navbar . . . 13

Step 9. My Quizzes (teacher list) . . . 13

Step 10. New Quiz form . . . 14

6. Build log, Part B: backend and database 15

Step 11. Check the backend tools . . . 15

Step 12. Create the backend and the first endpoint . . . 15

Step 13. PostgreSQL in Docker . . . 16

Step 14. Database tables with SQLAlchemy and Alembic . . . 16

Step 15. Login, JWT and test users . . . 17

Step 16. Quiz API . . . 18

Step 17. Questions and options API . . . 18

7. Build log, Part C: editor, student screens and results 19

Step 18. Quiz Editor page . . . 19

Step 19. Attempts API (start, submit, score) . . . 20
Step 20. Student pages . . . 20
Step 21. Teacher results page . . . 22
8. Testing **24**
8.1 Step 22: automated tests with pytest . . . 24
8.2 Test results . . . 24
8.3 Manual testing in the browser . . . 25
9. Problems and solutions (summary) **26**
10. Traceability and review against the SDD **27**
10.1 Brief requirements . . . 27
10.2 Functional requirements . . . 27
10.3 Final review . . . 27
11. How to run the project **28**
11.1 Ideas for later versions . . . 28
Appendix A. Project structure and data **29**
A.1 Folder structure . . . 29
A.2 Database tables . . . 29
A.3 API endpoints (18) . . . 29

1. Introduction and approach

1.1 The assignment

The task was to create a Specification Driven Development (SDD) document for a project of my own choice, and then build and test that project. The assignment gave a Ticketing System as the example, built with React JS for the UI, FastAPI for the API and PostgreSQL for the database, and it asked to test the application.

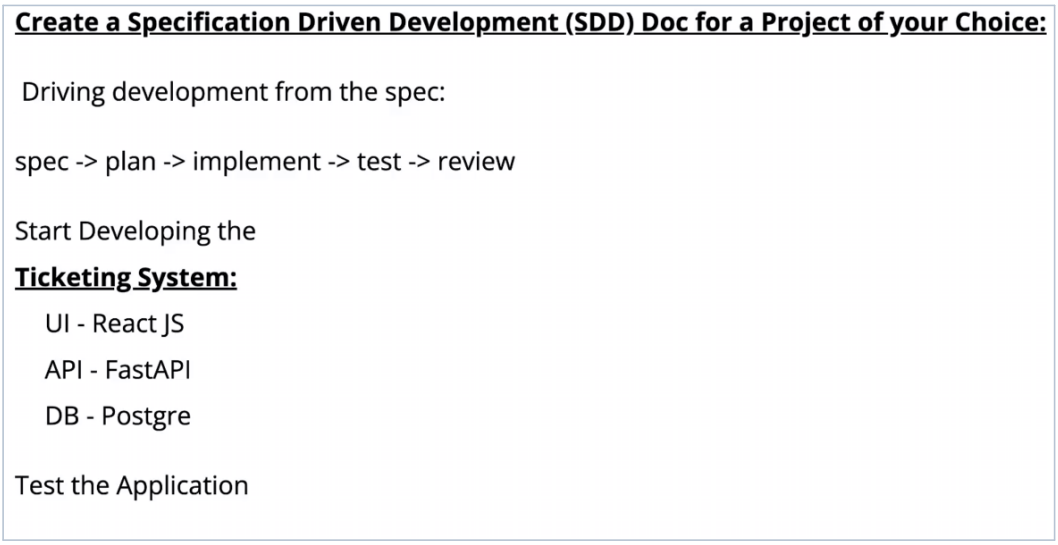


Figure 1. The assignment: create an SDD for a project of my choice; flow spec -> plan -> implement -> test -> review; stack React JS, FastAPI, PostgreSQL.

1.2 The development flow

The assignment describes one flow, and I followed it in order:

Phase	What I did in this project
Spec	Chose the project and wrote the SDD document (Section 3): requirements, rules, screens, API, data model, test cases.
Plan	Split the work into 22 small steps in a fixed order (Section 5 to 8), each tied to SDD requirements.
Implement	Built the React frontend, the FastAPI backend and the PostgreSQL database one step at a time.
Test	Wrote 30 automated tests that map to the SDD test cases (TC01 to TC31) and ran the application by hand.
Review	Compared the finished application against the SDD (Section 10) and listed decisions and problems (Section 9).

1.3 How I worked

I built the project together with an AI assistant, step by step. For every step the assistant explained what the step does and gave the code or commands. I typed or pasted each piece myself, ran it, checked the result on my own computer, and reported back with the output or a screenshot before moving on. The SDD was the single source of truth: a chat message never overrode it. When something failed, I sent the error text and we fixed the cause before continuing. Section 9 lists every problem found this way.

1.4 The reference document

I was given a reference SDD for the IT Helpdesk (a 13-page specification for a ticket system). I wanted my document to be written in the same way. From it I copied the structure and the writing rules: numbered sections, requirement IDs with a basis tag (S = from the brief, P = proposed), a permission table, a lifecycle table with allowed status changes, a REST contract, an error contract, quality requirements, mandatory test cases traced to requirements, a delivery plan, and a decision table for stakeholders.

2. Choosing the project

The first list of topics I had was: Task Management, Appointment Booking, Employee Leave Management, Game Booking, Task Allotment, Video Streaming, Interview Scheduling, AI Powered Bug Tracking, Library Management, Student Attendance Management, Smart Expense Anomaly Detector, Telecom Billing, Smart Spend Assurance, Financial Leakage Detection and Job Portal and Recruitment. I wanted a topic that was not on that list and not too close to the Ticketing System example.

I asked for projects that are easy to select and can be finished correctly. The shortlist of new topics was: Online Quiz and Exam System, Hostel Room Allocation, Blood Donation Management, Event Registration, Complaint Management (rejected: too close to a ticket system) and Gym Membership Management.

2.1 Decision: Online Quiz and Exam System

- It needs only 6 database tables: users, quizzes, questions, options, attempts and attempt answers.
- The business logic is small and clear: a score is the number of correct answers, and each student attempts each quiz once.
- Testing is easy to define: submit answers and check the score, try forbidden actions and check the error.
- It demonstrates well: a teacher creates a quiz, a student takes it, and both see results.
- It has two roles (Teacher and Student), so permissions are a real part of the specification.

2.2 Scope decision

To keep the project correct and finishable, the SDD excludes: timers, question types other than single-answer multiple choice, negative marking, question banks, retakes, email or push notifications, file uploads, registration screens, password reset, reports and charts, and multiple organizations. These exclusions are written in the SDD (Section 1.5) so that nothing outside the specification is built.

3. The specification (SDD)

The SDD document has 11 sections plus a short section 0 that explains how an AI coding agent must use it. It was delivered as a PDF in the same style as the helpdesk reference.

3.1 Structure compared to the reference

Reference (IT Helpdesk)	My SDD (Online Quiz and Exam System)
1-2 Scope, assumptions, actors and access	1 Scope and assumptions (S01-S08, A01-A05); 2 Actors and access control (Teacher, Student)
3 Functional requirements	3 Functional requirements FR01-FR14
4 Ticket lifecycle and business rules	4 Quiz lifecycle (Draft, Published, Closed) and attempt lifecycle (In Progress, Submitted)
5 Web interface specification	5 Screens and UI rules UI01-UI07
6 Data model and validation	6 Six tables, field rules and constraints
7 REST API contract	7 REST API under /api/v1
8 Errors, notifications, reliability	8 Error format, error codes, API01-API03
9 Quality and operational requirements	9 NF01-NF08 (security, performance, usability, consistency)
10 Acceptance criteria and test cases	10 Test cases TC01-TC31 traced to requirements
11 Delivery plan and decisions	11 Six delivery tasks, decision table, change control, review checklist

3.2 Key rules written in the SDD

- **Roles:** Teacher and Student. Accounts are created by a seed script; there is no registration screen (A01).
- **Quiz status:** every new quiz starts as Draft (S07). Draft can become Published (needs at least 1 valid question) and Published can become Closed. All other changes are rejected with 409 INVALID_TRANSITION.
- **Locking:** questions and details can only be changed while a quiz is Draft (409 QUIZ_LOCKED otherwise).
- **Questions:** 2 to 6 options with exactly one correct option; up to 50 questions per quiz.
- **Attempts:** one attempt per student per quiz, enforced by a database unique constraint (P08).
- **Scoring:** computed only on the server from stored correct flags; a client-sent score is rejected (P09).
- **Answer privacy:** correct answers are never sent to a student before that student submits (P04).
- **Not found rule:** a missing item and an item the user may not see both return the same 404 (P03).

3.3 Requirement groups

Group	IDs	Meaning
S	S01-S08	Requirements taken directly from the brief and the assignment
A	A01-A05	Proposed assumptions (one organization, statuses, one mark per question, one attempt, no timer)
P	P01-P10	Proposed rules: access, ownership, lifecycle, scoring, atomic commits
FR	FR01-FR14	Functional requirements for quizzes, questions, attempts and results
UI	UI01-UI07	Form behavior, empty/loading/error states, navigation, accessibility, dates
API	API01-API03	Concurrency, transactions, authorization order
NF	NF01-NF08	Security, secrets, performance, usability, consistency, logging, verification, maintainability
TC	TC01-TC31	Test cases, each traced to the requirements it proves

4. Tools and environment

Before writing any code I checked what was installed on the computer. The results were:

Tool	Version found	Used for
Node.js	v22.23.3	Runs the React development server
npm	10.9.9	Installs frontend packages
Python	3.14.7	Runs the FastAPI backend and the tests
pip	26.2.1	Installs backend packages
Docker Desktop	Docker 29.8.1, Compose v5.5.1	Runs PostgreSQL 16 in a container
psql	not installed	Not needed: the database commands run inside the container with `docker exec`
VS Code	installed	Editor and terminals

Main packages that were installed:

Area	Packages (versions from my installs)
Frontend	react 19.3.1, react-dom 19.3.1, vite 8.3.5, react-router-dom 7.18.5, axios 1.21.0, oxlint 1.90.0
Backend	fastapi, uvicorn 0.54.0, starlette 1.10.0, SQLAlchemy 2.1.6, alembic, psycopg (binary), pydantic with email support, python-jose, bcrypt, python-dotenv
Testing	pytest 9.2.0, httpx

4.1 Architecture

The application has three layers, as the SDD requires. The React app runs on port 5173 and talks to the FastAPI API on port 8000 over JSON. The API owns all permissions, validation, business rules and transactions. PostgreSQL runs in Docker and stores the data.

Layer	Port	Technology	Responsibility
Browser UI	5173	React JS (Vite)	Screens, form checks, role-based navigation
API	8000	FastAPI, SQLAlchemy	Login, roles, validation, scoring, transactions
Database	5433 (host)	PostgreSQL 16 in Docker	Persistent data, constraints, unique rules

5. Build log, Part A: the React frontend

The frontend was built first, as the project instructions required: create the project with the Vite command, then build the components, then connect everything to the API.

Step 1. Check Node.js

Goal: make sure the tools for React are installed.

```
node -v
npm -v
```

Result: `v22.23.3` and `10.9.9`. Both are fine (Node 18 or higher is needed), so no installation was required.

Step 2. Create the React app with Vite

Goal: create the project skeleton. I opened my project folder in VS Code and ran:

```
npm create vite@latest sdd -- --template react
```

The tool asked which linter to use and I chose **Oxlint**. It also asked to install and start the app, and I answered Yes. It installed 24 packages with 0 vulnerabilities and started the dev server at http://localhost:5173. The project folder is named `sdd` because that is the name I typed in the command.

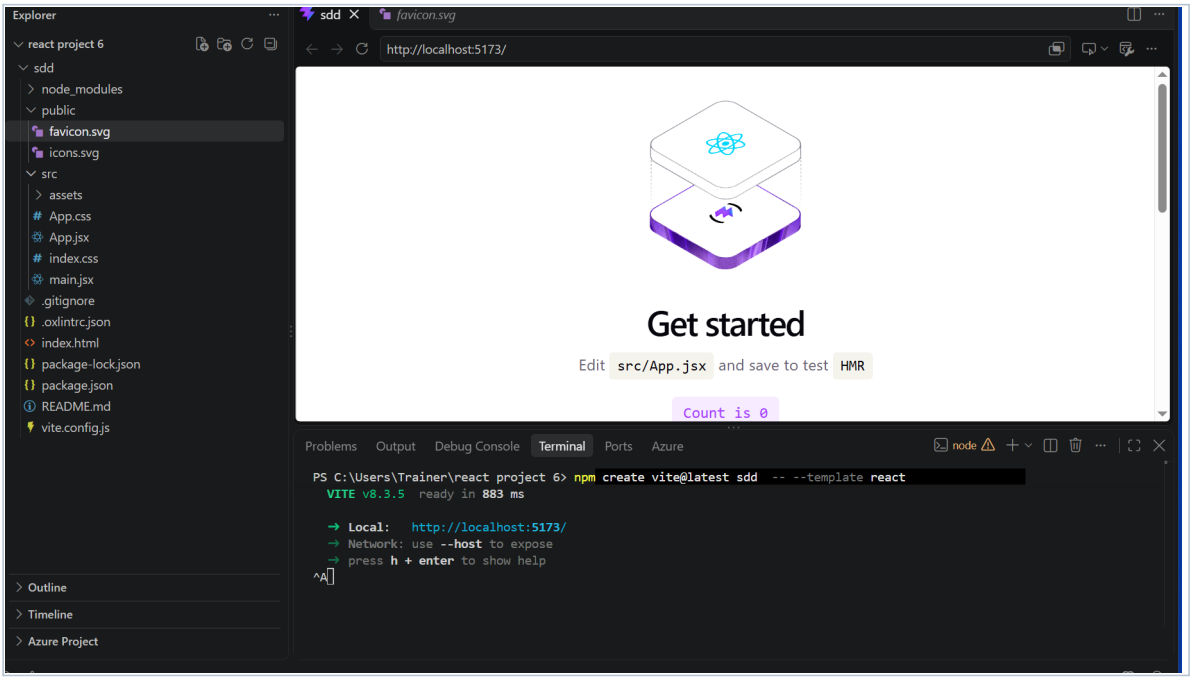


Figure 2. The new Vite + React project running: files in the left panel, the Get started page in the preview and the dev server in the terminal.

PROBLEM: The folder is called sdd, not frontend
My guide used the name frontend, but I had created the app as sdd. The terminal also showed a stray ^A, which is harmless.

SOLUTION: Use sdd everywhere
I kept the name sdd and used it in every later command (cd sdd, npm run dev) and in the README. The dev server was left running in its own terminal and I opened a second terminal for other commands.

Step 3. Extra packages and folders

Goal: add routing and HTTP libraries and create the SDD folder structure.

```
cd sdd
npm install react-router-dom axios
mkdir src\api, src\auth, src\components, src\pages
```

I also created a `.env` file in the sdd folder with the API address:

```
VITE_API_URL=http://localhost:8000/api/v1
```

`react-router-dom` provides pages and navigation, and `axios` is used for all API calls. The PowerShell output listed the four new folders (api, auth, components, pages) and the left panel showed them under src.

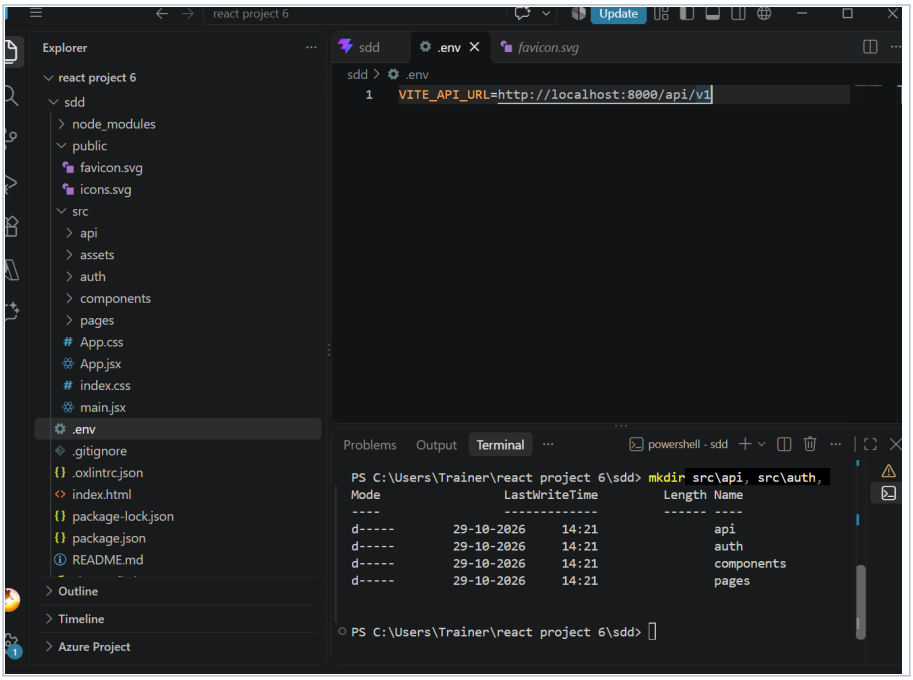


Figure 3. Folders api, auth, components and pages created under src, the .env file with VITE_API_URL, and the mkdir output in the terminal.

`npm list react-router-dom axios` confirmed react-router-dom 7.18.5 and axios 1.21.0 were installed.

Step 4. The API client (src/api/client.js)

Goal: one place that talks to the backend. It reads the address from `.env`, adds the login token to every request, and handles an expired login in one place (SDD UI04). The file has the extension `.js` because it has no JSX tags; files that contain tags (components and pages) use `.jsx`.

Key part of client.js (final version)

```
client.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

client.interceptors.response.use(
  (response) => response,
  (error) => {
    const isLogin = error.config?.url?.includes("/auth/login");
    if (error.response && error.response.status === 401 && !isLogin) {
      localStorage.removeItem("token");
      localStorage.removeItem("user");
      window.location.href = "/login";
    }
    return Promise.reject(error);
  }
);
```

The final version skips the redirect for the login request itself. Without that rule, a wrong password would return 401, the page would reload, and the user would never see the message "Wrong email or password".

Step 5. AuthContext (who is logged in)

Goal: remember the logged-in user and their role on every page. `src/auth/AuthContext.jsx` provides `user`, `login(email, password)` and `logout()`. `login` calls `POST /auth/login`, saves the token and the user in localStorage, and updates the state. The saved user keeps the login after a page refresh.

Step 6. Router, protected routes and the home redirect

Goal: wire the pages together and enforce roles in the browser. I replaced `main.jsx` and `App.jsx` and created `src/auth/ProtectedRoute.jsx`.

- `BrowserRouter` enables pages; `AuthProvider` makes the user available everywhere.
- `ProtectedRoute` sends a visitor who is not logged in to /login, and shows "This action is not available." when the role is wrong (SDD P01, P03, UI04).
- The home route sends a Teacher to /teacher/quizzes and a Student to /student/quizzes.

Check: opening http://localhost:5173 jumped to /login, and typing /teacher/quizzes in the address bar also returned to /login. This proved the protection worked.

Step 7. The Login page

Goal: the first real screen. `src/pages/Login.jsx` follows the SDD form rules: labelled fields (UI01), whitespace-only input is rejected, the button is disabled while the request runs (UI02), and server errors are shown without technical details (UI03). I also replaced `index.css` with simple styles and emptied `App.css`.

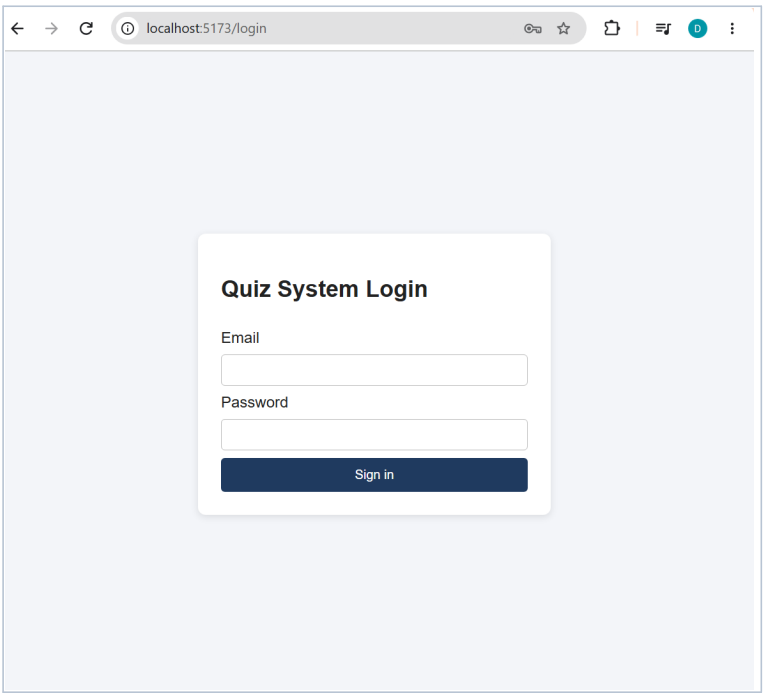


Figure 4. The login page at localhost:5173/login (final version, with the Quiz System Login card).

Check: clicking Sign in with empty fields showed "Email and password are required." With a test email and password and no backend yet, the page showed "Cannot reach the server. Please try again.", which was the correct result at that time.

PROBLEM: Console errors and the DevTools paste block
While testing, the console showed ERR_CONNECTION_REFUSED for /auth/login (expected, the backend did not exist). When I tried to paste a test command into the console, Chrome blocked it with a warning, and typing allow pasting produced Uncaught SyntaxError: Unexpected identifier 'pasting'.

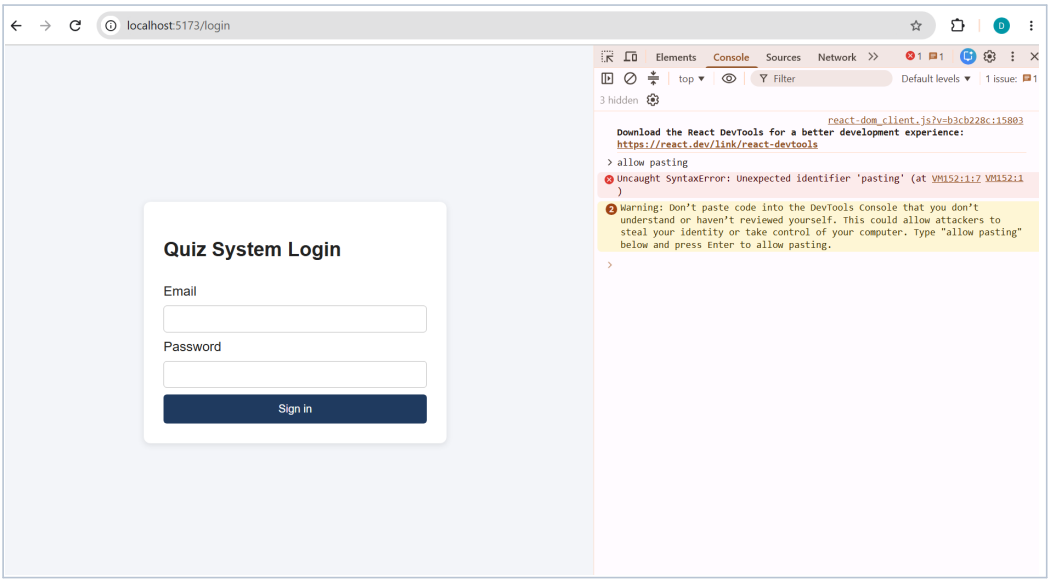


Figure 5. The login page with DevTools open: the paste warning and the SyntaxError from typing allow pasting.

SOLUTION: Type the line by hand or use the Application tab
The block only applies to pasting. I typed the test line by hand (or added the key in DevTools, Application, Local Storage, key user). The connection errors were ignored because they were expected until the backend was running.

Step 8. Layout with the navbar

Goal: one top bar shared by all logged-in pages. `src/components/Layout.jsx` shows different links per role (SDD UI05): a Teacher sees My Quizzes and New Quiz, a Student sees Available Quizzes and My Results. No menu item exists for an excluded feature such as Reports. `<Outlet />` is the empty area where each page appears. `App.jsx` now has two route groups, one wrapped in `ProtectedRoute role="Teacher"` and one for `Student`.

Check (before the backend existed): I faked a login by adding a user entry in localStorage. The blue bar appeared with My Quizzes highlighted, New Quiz, the user name and a Logout button; /student/quizzes showed "This action is not available."

PROBLEM: The fake login disappeared
After I clicked Logout and later wanted to open My Quizzes again, the fake login was gone.

SOLUTION: Logout clears the login by design
Logout removes the token and user from localStorage. For testing I added the fake user again, or avoided Logout until the real login existed. This problem disappeared after Step 15, when real accounts were available.

Step 9. My Quizzes (teacher list)

Goal: the list of the teacher's quizzes. `src/pages/MyQuizzes.jsx` calls `GET /quizzes`, supports a Status filter (All, Draft, Published, Closed), and shows loading, empty ("No quizzes found.") and error states with a Retry button (SDD UI03). Status is written as text (UI06) and dates use the browser time zone (UI07).

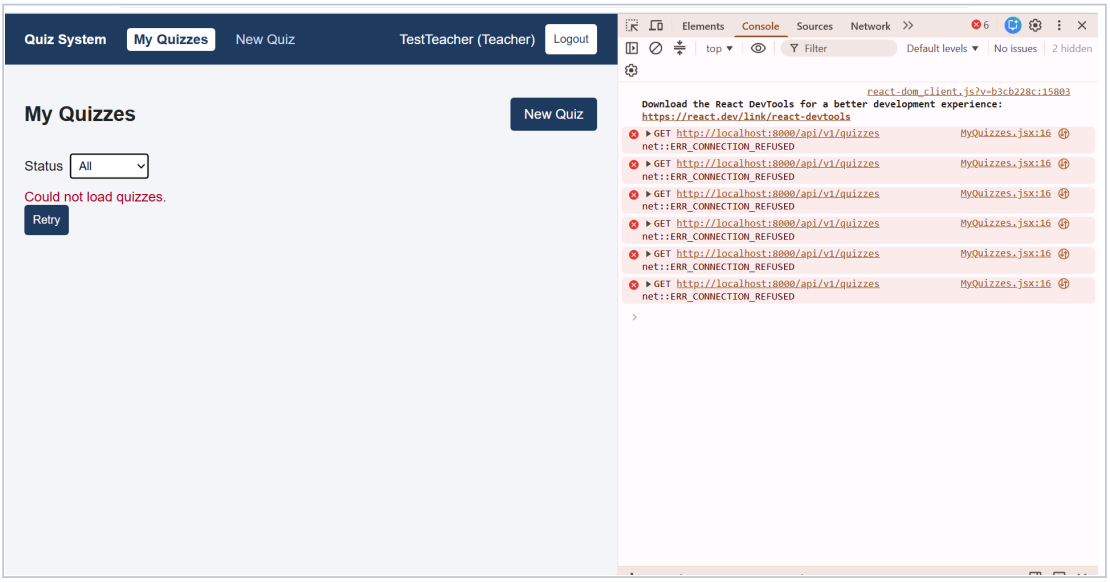


Figure 6. My Quizzes before the API existed: Could not load quizzes with a Retry button, and ERR_CONNECTION_REFUSED lines in the console. This was the expected result at this step.

The red console lines repeat six times because the page reloads on every open, filter change and Retry click, and React runs effects twice in development.

Step 10. New Quiz form

Goal: create a quiz (SDD FR01, FR02). `src/pages/NewQuiz.jsx` checks the rules on the client: title 5 to 150 characters and description 10 to 1,000 characters, after trimming spaces. The typed text stays in the boxes when an error happens, and the button is disabled while saving.

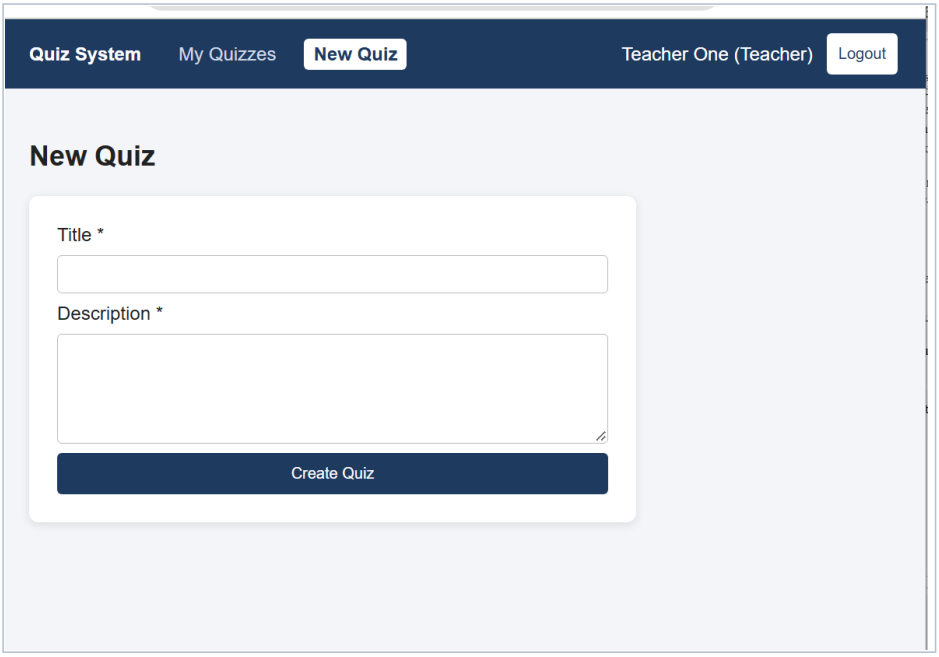


Figure 7. The New Quiz form: Title and Description with required markers and the Create Quiz button.

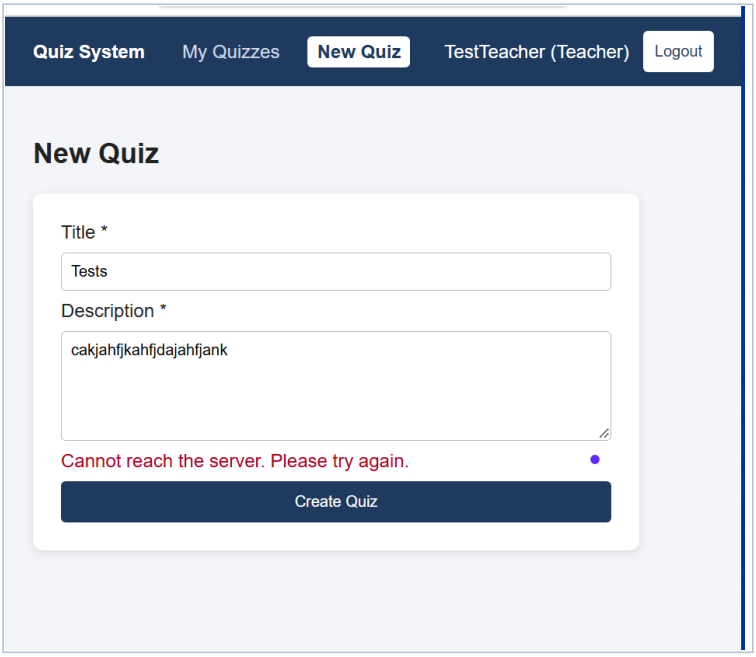


Figure 8. Validation test: the title Tests (5 letters) and a description pass the checks. With no backend yet the form correctly showed Cannot reach the server and kept my text.

Check: an empty form showed both error messages; a 4-letter title still failed and a 5-letter title passed. This matched the SDD boundary rule (TC03).

6. Build log, Part B: backend and database

Step 11. Check the backend tools

```
python --version
pip --version
docker --version
psql --version
```

Python 3.14.7, pip 26.2.1 and Docker 29.8.1 were present. `psql` was not found, which is fine because PostgreSQL runs inside Docker and I can run `psql` in the container.

A question I had at this point was where to run a command such as `docker exec quiz-db psql ...`. The answer is: in any free terminal in VS Code. It does not need the virtual environment or a particular folder.

Step 12. Create the backend and the first endpoint

Goal: a running FastAPI app with a health check. I created a `backend` folder, a Python virtual environment, installed the packages and created the layered folders from the SDD (core, models, schemas, routers, services, repositories, tests).

```
cd backend
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install fastapi "uvicorn[standard]" sqlalchemy alembic "psycopg[binary]" "pydantic[email]" python-
jose bcrypt python-dotenv pytest httpx
mkdir app, app\core, app\models, app\schemas, app\routers, app\services, app\repositories, tests
```

`app/main.py` created the FastAPI app with CORS restricted to http://localhost:5173 (SDD NF01) and one endpoint, `GET /api/v1/health`. The API starts with:

```
uvicorn app.main:app --reload --port 8000
```

Check: http://localhost:8000/api/v1/health returned the status ok message, and http://localhost:8000/docs showed the automatic API documentation. Back in React, the My Quizzes error changed from ERR_CONNECTION_REFUSED to **404 Not Found**. The uvicorn log showed `GET /api/v1/quizzes ... 404 Not Found`, which proved the frontend now reached the backend and only the quizzes endpoint was missing.

PROBLEM: Commands got mixed together in PowerShell
I ran the folder, venv and activate commands as a block. The terminal text got mixed (for example venv\Scripts\v venv) and the virtual environment was not created. The prompt was still in react project 6 and not in backend.

SOLUTION: One command at a time, then verify
I went into the folder with cd backend, then ran python -m venv venv, then .\venv\Scripts\Activate.ps1 as separate commands, waiting for the prompt each time. The check python -c "import sys; print(sys.prefix)" printed a path ending in backend\venv, and the prompt showed (venv).

The package list was installed successfully on Python 3.14 (pip list showed SQLAlchemy 2.1.6, starlette 1.10.0, uvicorn 0.54.0 and the others), so no compatibility problem appeared.

Step 13. PostgreSQL in Docker

Goal: a real database. I started Docker Desktop, checked it with `docker info`, and created `docker-compose.yml` for PostgreSQL 16 (user quiz, database quizdb, a named volume for the data). Then:

```
docker compose up -d db
docker compose ps
docker exec quiz-db psql -U quiz -d quizdb -c "select 1;"
```

The image postgres:16 was downloaded, the container quiz-db showed Up, and `select 1` returned one row. I also created `backend/.env` with DATABASE_URL, JWT_SECRET and JWT_EXPIRE_MINUTES (secrets stay out of the code, SDD NF02).

PROBLEM: Windows Defender Firewall popup
When Docker started for the first time, Windows showed: Windows Defender Firewall has blocked some features of Docker Desktop Backend.

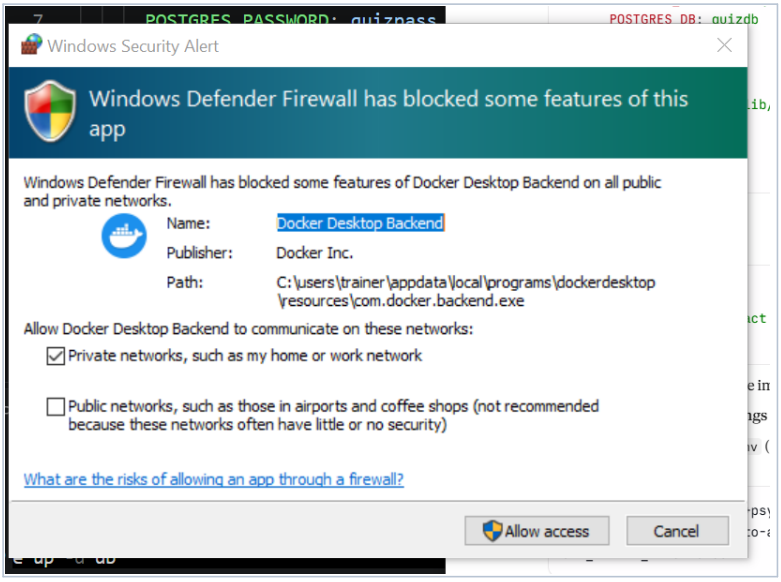


Figure 9. The Windows Security Alert for Docker Desktop Backend.

SOLUTION: Allow private networks only
This is a normal prompt. I left Private networks ticked, left Public networks unticked and clicked Allow access. The image download then continued and finished in about 23 seconds.

Step 14. Database tables with SQLAlchemy and Alembic

Goal: create the six tables of SDD Section 6. I wrote the settings file (`core/config.py`), the connection (`core/database.py`) and the models (`models/models.py`), then set up Alembic and ran the migration:

```
alembic init alembic
# replace alembic/env.py with the project version (reads DATABASE_URL, loads the models)
alembic revision --autogenerate -m "create tables"
alembic upgrade head
```

The rules of the SDD are written as database constraints, not only as code:

SDD rule	Constraint in the database
One attempt per student per quiz (P08)	UNIQUE (quiz_id, student_id) on attempts
Exactly one correct option per question	Partial unique index on options where is_correct is true
Valid roles and statuses	CHECK constraints on users.role, quizzes.status, attempts.status
Unique email, unique answer per question	UNIQUE on users.email and on (attempt_id, question_id)

SDD rule	Constraint in the database
Deleting a quiz removes its questions and options	ON DELETE CASCADE foreign keys
Fast lists	Indexes on quizzes(teacher_id, status), attempts(student_id), attempts(quiz_id)

PROBLEM: password authentication failed for user quiz
alembic revision stopped with a long traceback ending in: FATAL: password authentication failed for user quiz (connection to 127.0.0.1 and ::1 on port 5432). The same password had worked inside the container a moment earlier.

Investigation: the command `netstat -ano | findstr :5432` showed two different processes listening on port 5432 (process IDs 1772 and 12488). One was Docker and the other was a PostgreSQL that was already installed on this Windows computer. My application was connecting to that other PostgreSQL, which has no user called quiz. The earlier select 1 test could not detect this because it ran inside the container.

SOLUTION: Move our database to port 5433
I changed the compose file port mapping from 5432:5432 to 5433:5432, changed DATABASE_URL in backend/.env to use port 5433, ran docker compose up -d db again (docker compose ps showed 5433->5432), and repeated alembic revision and alembic upgrade head. The other PostgreSQL was left running because it is not part of this project.

Check: `docker exec quiz-db psql -U quiz -d quizdb -c "\dt"` listed 7 tables: alembic_version, attempt_answers, attempts, options, questions, quizzes and users.

Step 15. Login, JWT and test users

Goal: make the Login page work. I added the following files:

File	Purpose
core/errors.py	The SDD error format {error: {code, message, fields, request_id}} and handlers for validation, 401, 403, 404
core/security.py	bcrypt password hashing and JWT creation and checking (60-minute expiry)
schemas/auth.py	Login request; unknown fields are rejected
core/deps.py	get_current_user and require_role; the role and active status always come from the database (SDD P01)
routers/auth.py	POST /auth/login and GET /auth/me
seed.py	Creates Teacher One, Student One and Student Two (skips accounts that already exist)

I ran `python seed.py` and saw three Created lines. In React I fixed client.js so a wrong password shows its message (see Step 4).

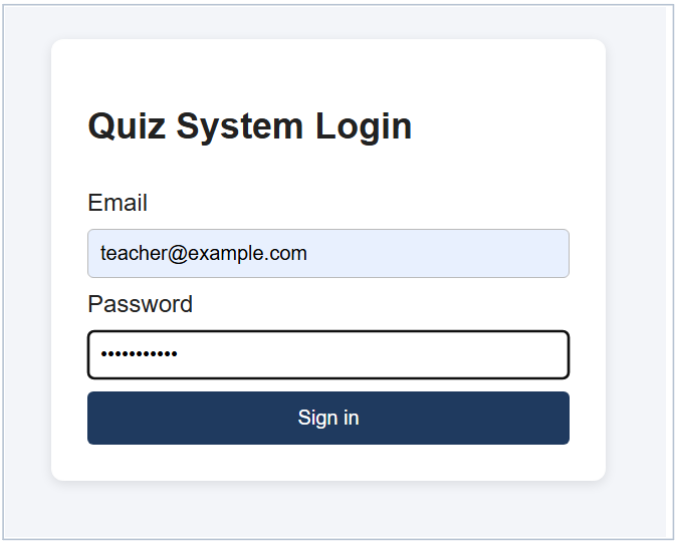


Figure 10. Signing in as the Teacher (email filled, password hidden).

Check: a wrong password showed "Wrong email or password." and stayed on the login page; the teacher account landed on My Quizzes with "Teacher One (Teacher)" in the bar; the student account landed on Student: Available Quizzes. I confirmed all three behaviours worked.

New students: there is no sign-up screen by design (SDD A01). To add a student I add one line to the USERS list in seed.py and run python seed.py again; existing users are skipped.

Step 16. Quiz API

Goal: the quiz endpoints (FR01 to FR06, P02, P06, P07). Files: `schemas/quiz.py`, `routers/quizzes.py`, and the router registered in `main.py`.

Endpoint	Behaviour
POST /quizzes	Teacher only. Creates a quiz with status Draft. Extra fields such as status or teacher_id are rejected with 422.
GET /quizzes	Teacher: own quizzes (optional status and title search). Student: Published quizzes only, with an attempted flag.
GET /quizzes/{id}	Teacher owner: full quiz with correct answers. Student: Published summary only, no questions.
PATCH /quizzes/{id}	Draft only; otherwise 409 QUIZ_LOCKED.
DELETE /quizzes/{id}	Draft only; otherwise 409.
POST /quizzes/{id}/publish	Needs at least 1 question and every question valid, otherwise 422; only from Draft.
POST /quizzes/{id}/close	Only from Published; otherwise 409 INVALID_TRANSITION.

A teacher can only see and change their own quizzes. Someone else's quiz returns the same 404 as a missing one, so its existence is not revealed (P03).

Check: after the reload, My Quizzes changed from the error to "No quizzes found." I created Python Basics with New Quiz, and the database query `select title, status from quizzes;` returned `Python Basics | Draft`.

Step 17. Questions and options API

Goal: add, replace and remove questions on a Draft quiz (FR03, P07). `schemas/question.py` validates 2 to 6 options and exactly one correct option. `routers/questions.py` returns 409 QUIZ_LOCKED when the quiz is not Draft, and the authorization check happens before the lock check. A quiz can have at most 50 questions. Replacing a question removes the old options first, so two correct rows never exist together.

I tested it through the automatic documentation page: logging in, authorizing with the token, getting the quiz ID and posting a valid question.

PROBLEM: The manual API test needed many ID copies
To start an attempt I would have to copy a quiz id, an attempt id, question ids and option ids between calls in the docs page. I was confused about where the attempt id appears, and asked whether all the manual checks were compulsory.

SOLUTION: Skip the manual checks and use automated tests
The attempt id is the first id at the top of the start response, but the checks were not compulsory. The student pages and the automated test suite check the same behaviour, so I moved on and wrote pytest tests in Step 22.

7. Build log, Part C: editor, student screens and results

Step 18. Quiz Editor page

Goal: manage a quiz from the screen instead of the docs page (FR03 to FR06, UI01 to UI04). I created `components/QuestionForm.jsx` (question text, 2 to 6 options, a radio button for the correct one, add and remove option) and `pages/QuizEditor.jsx`, added the route, and added styles. The editor shows the status badge; for a Draft it has Publish, Delete quiz, a details form and Add, Edit and Delete for questions; for a Published quiz everything becomes read-only and Close quiz and View results appear.

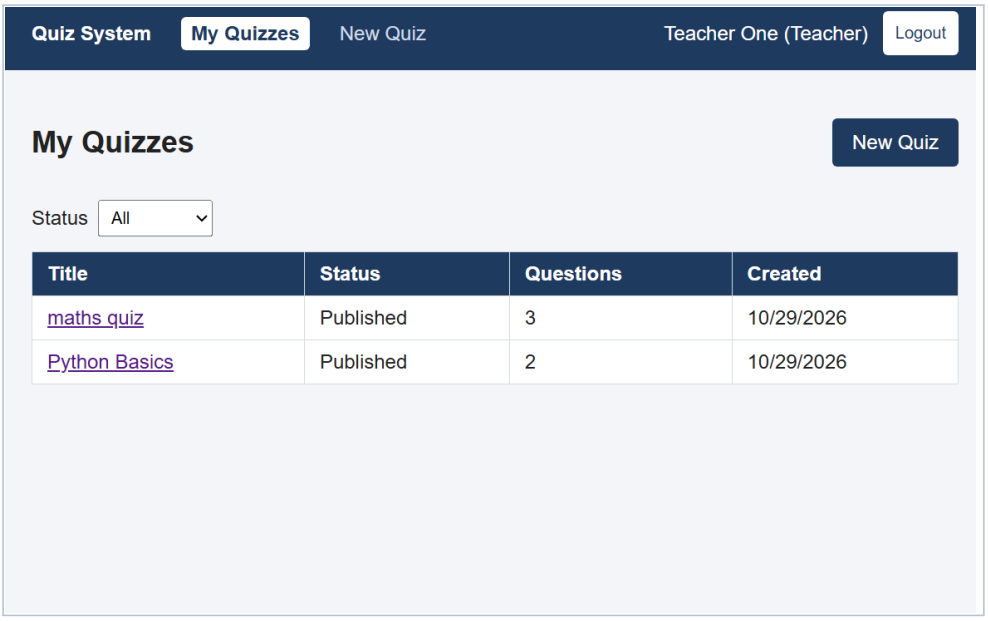


Figure 11. My Quizzes with real data: maths quiz (3 questions) and Python Basics (2 questions), both Published.

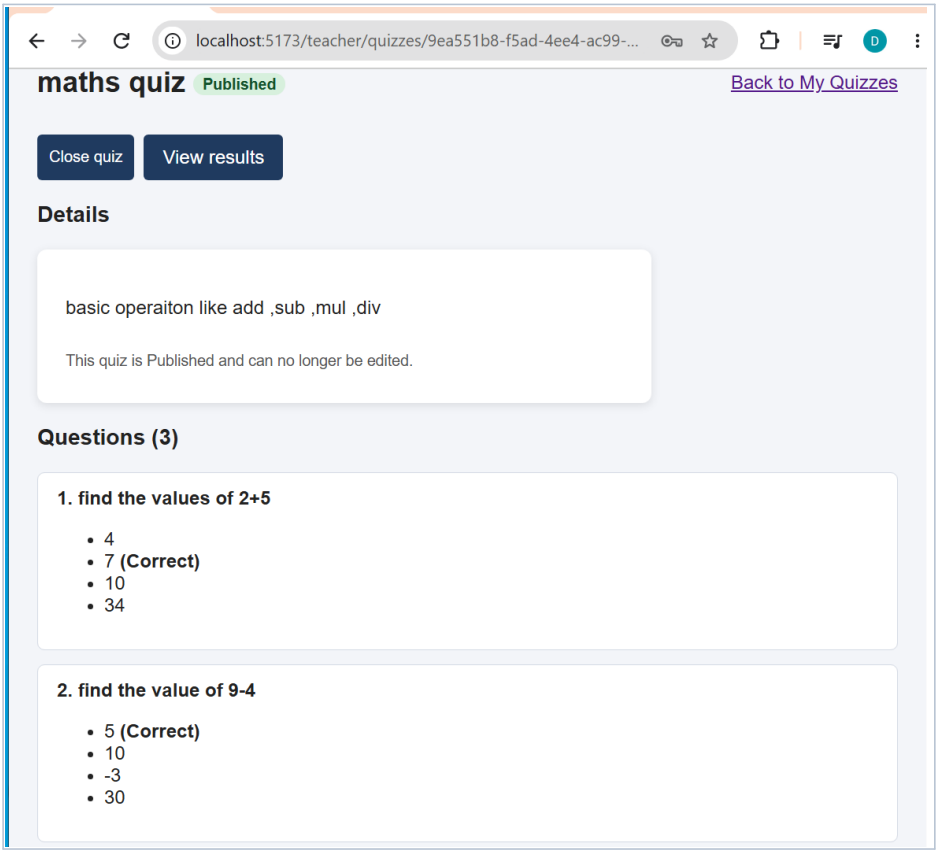


Figure 12. Quiz Editor for the Published maths quiz: green Published badge, Close quiz and View results buttons, read-only details with the message that the quiz can no longer be edited, and the questions with the correct option marked.

Checks I did: empty question text and empty option text each showed a red message and saved nothing; Publish with 0 questions showed "Add at least 1 question before publishing." and the quiz stayed Draft; after publishing, the Add, Edit and Delete buttons disappeared; Close quiz changed the badge to Closed. Creating a new quiz now opens its editor instead of a blank page.

Step 19. Attempts API (start, submit, score)

Goal: the core of the system (FR07 to FR14, P04, P08 to P10). Files: `schemas/attempt.py` and `routers/attempts.py`.

Endpoint	Behaviour
POST /quizzes/{id}/attempts	Student only. Quiz must be Published. A second attempt returns 409 ATTEMPT_EXISTS. The response contains questions without any is_correct field.
POST /attempts/{id}/submit	Locks the attempt row, validates every answer (question belongs to the quiz, option belongs to the question), computes the score on the server, and commits answers, score and status together.
GET /attempts	A student's own attempts, newest first (optional status filter).
GET /attempts/{id}	Student: own attempt only. Submitted: per-question review with correct answers. In Progress: questions again, so the quiz can be resumed after a refresh.
GET /quizzes/{id}/results	Teacher owner only: attempt count, average percentage, highest and lowest score, and the list of attempts.

Key part of submit_attempt

```
# the scoring rule (server side only)
score = 0
for question_id, question in questions.items():
    selected = chosen.get(question_id)
    db.add(AttemptAnswer(attempt_id=attempt.id, question_id=question_id, option_id=selected))
    if selected is not None and any(o.id == selected and o.is_correct for o in question.options):
        score += 1
total = len(questions)
percentage = round(score * 100 / total, 1) if total else 0.0
```

Safety rules: a repeated submit returns 409 ATTEMPT_SUBMITTED and never changes the stored score; two simultaneous start requests create exactly one attempt because the database unique constraint decides; a client-sent score field is rejected as an unknown field.

Step 20. Student pages

Goal: the four student screens. I created `AvailableQuizzes.jsx`, `TakeQuiz.jsx`, `ResultDetail.jsx` and `MyResults.jsx`, and replaced the placeholders in `App.jsx`.

Page	What it does
Available Quizzes	Lists Published quizzes with question count and description. Shows Start, or an Attempted badge with a View button if the student already attempted it (FR07).
Take Quiz	Loads the attempt with radio buttons. If questions are unanswered it asks for confirmation before submitting. After submit it opens the result. Refreshing the page in the middle of a quiz keeps the quiz available.
Result Detail	Score, percentage, submit time and a review: each question is Correct, Incorrect or Not answered, with (Your answer) and (Correct answer) marked (FR12).
My Results	All submitted attempts of the student, newest first.

The student used here is Student Two. The login page and the pages after login looked like this:

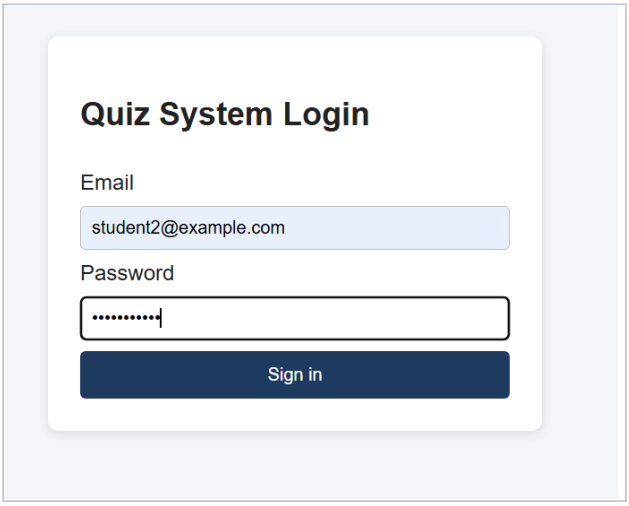


Figure 13. Signing in as Student Two.

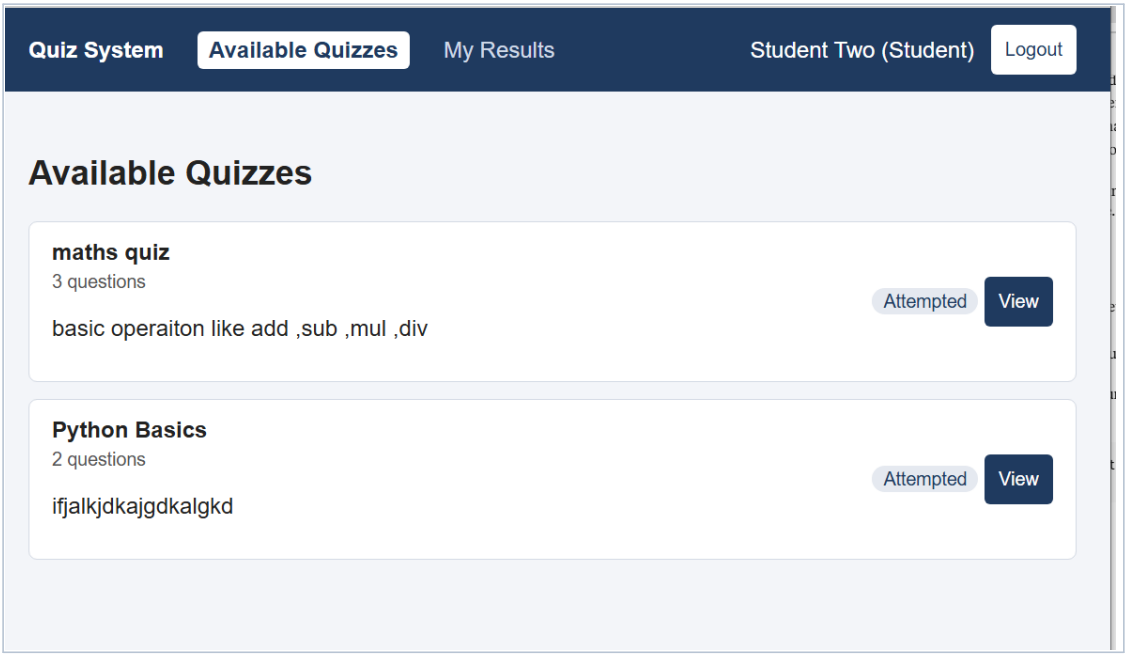


Figure 14. Available Quizzes for Student Two. Both quizzes already show the Attempted badge and a View button, because the student has submitted both.

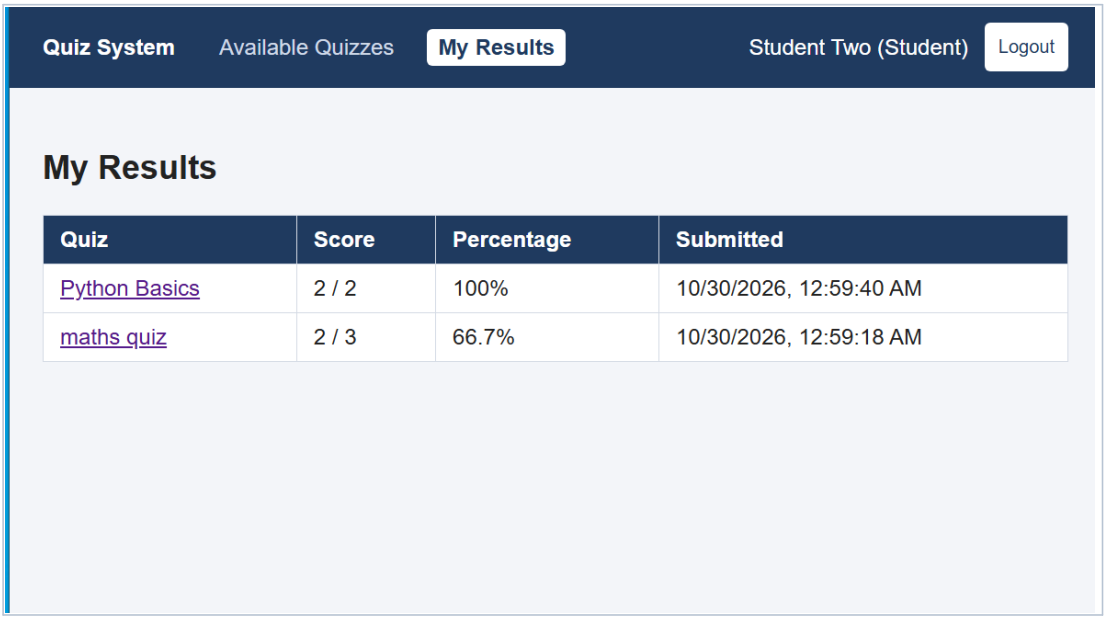


Figure 15. My Results for Student Two: Python Basics 2 / 2 (100%) and maths quiz 2 / 3 (66.7%), each with the submit time.

Checks I did: Start opened the quiz; submitting with one unanswered question asked for confirmation; the result page showed the score and the review; the attempted quiz could not be started a second time; a different student (Student Two) still saw Start for quizzes attempted by Student One, because attempts belong to each student.

Step 21. Teacher results page

Goal: let the teacher see how students did (FR13, TC24). `pages/QuizResults.jsx` calls `GET /quizzes/{id}/results` and shows four boxes (Attempts, Average, Highest score, Lowest score) and a table of students with score, percentage and time, with paging when there are more than 20. I also added a **Results** column with a View link in My Quizzes (a dash for Drafts) and a View results button in the editor.

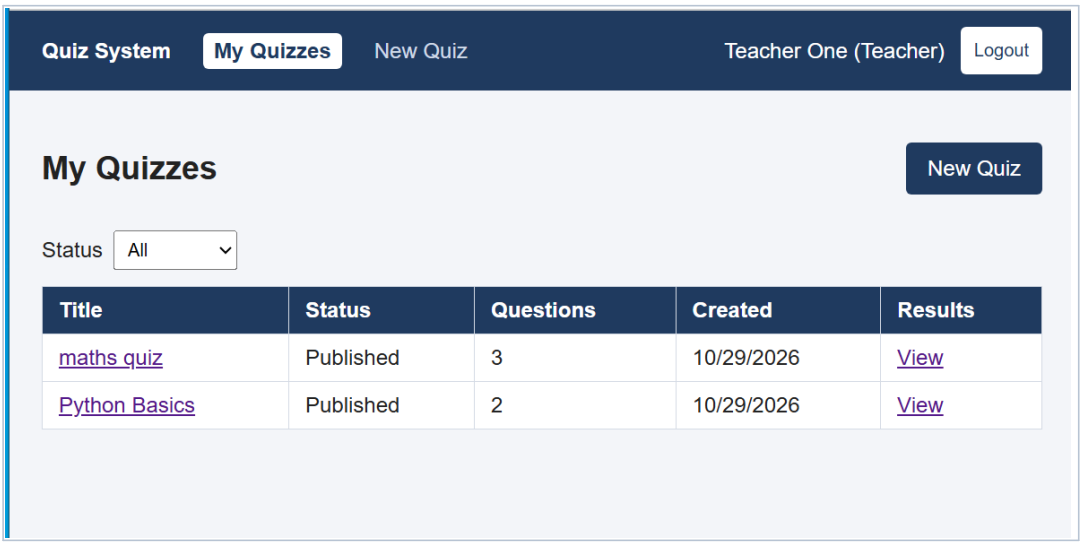


Figure 16. My Quizzes with the new Results column: View links for Published quizzes.

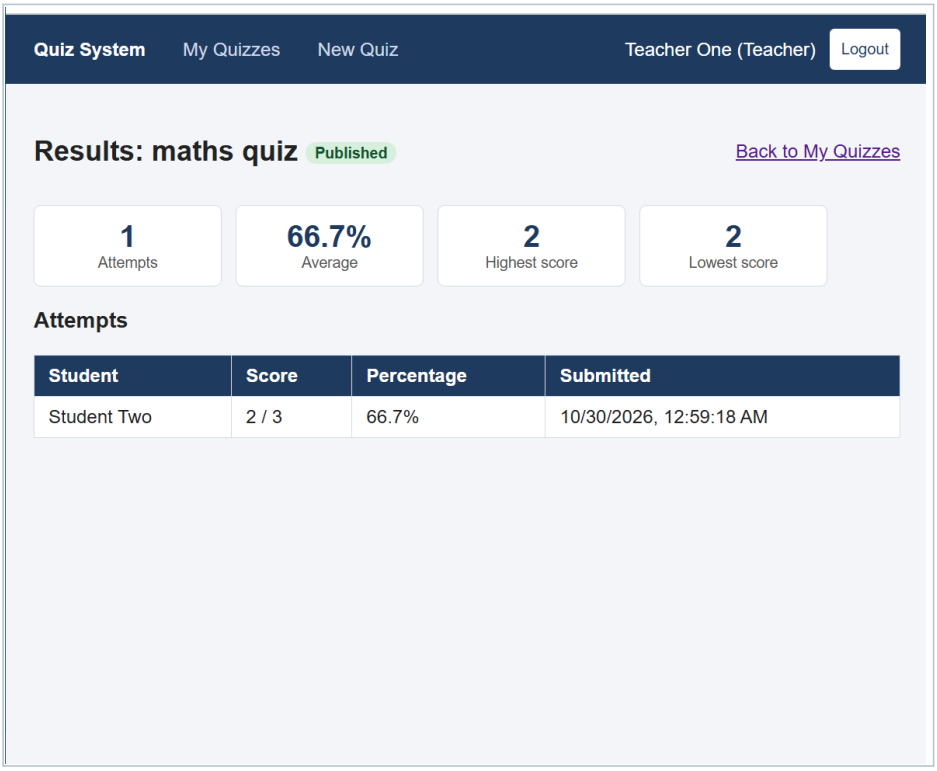


Figure 17. Results for maths quiz: 1 attempt, average 66.7%, highest and lowest score 2, and Student Two with 2 / 3 (66.7%). These numbers match the student's own result in Figure 15.

PROBLEM: The Results column was misaligned
I added the Results header cell in MyQuizzes.jsx, but the table body rows still had only four cells, so the new column was empty and the layout was wrong. I had also asked where exactly to put the new cell code.

SOLUTION: Add the cell inside each row, after the created date
Each row needs a fifth td after the cell that shows toLocaleDateString(): a dash for Draft, otherwise a View link to the results page. I replaced the whole file with the complete version so the header and body matched, and the column showed View for Published quizzes (Figure 16).

8. Testing

8.1 Step 22: automated tests with pytest

Goal: the assignment says to test the application, and the SDD defines test cases TC01 to TC31. I wrote 30 automated tests and left only TC30 (keyboard-only and narrow-screen check) as a manual test.

- The tests run against a **separate database**, quizdb_test, which is wiped before every test, so my real quizzes are never touched. I created it with `CREATE DATABASE quizdb_test;`.
- `tests/conftest.py` switches the database URL to the test database, creates the tables, and seeds Teacher A and B, Student A, B and C and an inactive user.
- `tests/test_api.py` contains one test per test case; each test name carries its SDD ID, for example test_tc18_scoring.
- `pytest.ini` sets the test folder. A hook in conftest.py writes the result table to TEST_RESULTS.md after every run.
- Two tests are special: TC27 starts two attempts at the same moment with two threads and expects one 201 and one 409; TC28 forces a database failure during submit and checks that no answers or score remain.

```
cd backend
.\venv\Scripts\Activate.ps1
python -m pytest -v
```

PROBLEM: No module named pytest
I ran python -m pytest in a normal terminal and got: C:\Python314\python.exe: No module named pytest. The prompt began with PS and not (venv) PS.

SOLUTION: Activate the virtual environment first
pytest was installed inside the venv, not in the global Python. After running .\venv\Scripts\Activate.ps1 inside the backend folder, python -m pytest -v worked.

Result: `30 passed, 1 warning in 89.98s`. The single warning is a deprecation notice about the httpx test client and does not affect the results.

8.2 Test results

Test	Scenario	SDD trace	Result
TC01	Teacher creates a quiz (status Draft)	FR01, S07	PASS
TC02	Blank title, short description or unknown field rejected; nothing created	FR02	PASS
TC03	Title and description length boundaries (4/5/150/151 and 9/10/1000/1001)	FR02, Section 6	PASS
TC04	No token and inactive user rejected (401)	P01, P05	PASS
TC05	Student cannot create a quiz (403)	P02	PASS
TC06	Another teacher gets 404 on read, edit, results, publish	P02, P03	PASS
TC07	Teacher adds a question with 3 options	FR03	PASS
TC08	1 option, 7 options, 0 correct, 2 correct all rejected (422)	FR03	PASS
TC09	Publishing a quiz with 0 questions rejected	FR06	PASS
TC10	Publishing a valid quiz works	FR06	PASS
TC11	Published quiz is locked (409 QUIZ_LOCKED)	P07, FR04	PASS
TC12	Delete a Draft works; delete a Published returns 409	FR05	PASS
TC13	Student lists Published quizzes only, with attempted flag	FR07	PASS

Test	Scenario	SDD trace	Result
TC14	Student cannot read a Draft quiz (404)	P02	PASS
TC15	Start attempt: correct answers hidden in the response	FR08, P04	PASS
TC16	Second attempt rejected (409 ATTEMPT_EXISTS), one row stored	P08	PASS
TC17	Cannot start a Closed or Draft quiz	FR14	PASS
TC18	Scoring: 3 correct of 5 gives 3 / 5 and 60.0%	FR09, FR10	PASS
TC19	Option from another question or unknown question rejected	FR09	PASS
TC20	Submitting twice returns 409; score unchanged	FR11, API02	PASS
TC21	A client-sent score field is rejected	P09	PASS
TC22	Another student cannot read my attempt (404)	P02	PASS
TC23	Review after submit shows correct and incorrect answers	FR12	PASS
TC24	Teacher results: count, average, highest, lowest	FR13	PASS
TC25	Closing stops new attempts, existing results stay	FR14	PASS
TC26	Invalid status changes (close a Draft, publish a Closed)	P06	PASS
TC27	Two simultaneous starts create exactly one attempt	API01	PASS
TC28	Database failure during submit rolls everything back	P10, NF05	PASS
TC29	Script text is stored and returned as plain text	NF01	PASS
TC31	End-to-end: teacher builds, student attempts, teacher sees results	S01 to S04	PASS
TC30	Keyboard-only use and 375 px width for both journeys	UI06, NF04	Manual

8.3 Manual testing in the browser

Besides the automated tests I used the application as a real user. The screenshots in Part C are the evidence: a Teacher created and published quizzes; Student Two attempted both quizzes (Figures 14 and 15); the same scores appeared on the Teacher's results page (Figure 17), which shows the whole chain from quiz creation to results works end to end.

9. Problems and solutions (summary)

This table collects every problem from the build in one place.

#	Problem	Cause	Solution
1	Project folder is called sdd, not frontend	I typed sdd as the name in the Vite command	Used sdd in all commands and in the README
2	DevTools blocked pasting; typing allow pasting gave SyntaxError	Chrome protection against pasting into the console	Typed the line by hand, or used Application, Local Storage
3	Fake test login lost after Logout	Logout clears the saved user by design	Added it again for testing; real accounts replaced it in Step 15
4	ERR_CONNECTION_REFUSED on login and quizzes	The backend was not built yet (expected)	Built the API; the error became 404, then real data
5	Virtual environment not created	Several commands got mixed together in one paste	Ran cd backend, python -m venv venv and Activate.ps1 one by one and verified sys.prefix
6	Windows Firewall popup for Docker	First start of Docker Desktop Backend	Allowed private networks only
7	password authentication failed for user quiz	Another PostgreSQL on Windows already used port 5432 (netstat showed two listeners)	Moved Docker to port 5433 and updated docker-compose.yml and backend/.env
8	Wrong password message never appeared	The 401 handler reloaded the page for the login request too	client.js skips the redirect for /auth/login
9	Manual API checks needed many copied IDs	Docs page needs attempt, question and option IDs	Skipped; used the UI and automated tests instead
10	Results column misaligned in My Quizzes	Header cell added but the cell in each row was missing	Added the td inside each row; replaced the whole file
11	No module named pytest	Virtual environment was not active	Activated venv, then python -m pytest -v
12	psql command not found	psql is not installed on Windows	Used docker exec quiz-db psql inside the container

What I learned: most problems came from the environment (ports, virtual environments, terminals) and not from the application code. Verifying every step with a small check (a URL, a database query, a log line) found each problem right where it started.

10. Traceability and review against the SDD

10.1 Brief requirements

ID	Requirement	Implemented in	Verified by
S01	Teachers create quizzes with questions	routers/quizzes.py, questions.py; NewQuiz, QuizEditor, QuestionForm	TC01, TC07, TC08, TC31
S02	Students view and attempt published quizzes	routers/attempts.py; AvailableQuizzes, TakeQuiz	TC13 to TC17
S03	Automatic scoring	submit_attempt in attempts.py	TC18, TC20, TC21
S04	Students see results; teachers see results of their quizzes	ResultDetail, MyResults, QuizResults	TC23, TC24, TC31
S05	Validation and permissions enforced	Pydantic schemas, require_role, ProtectedRoute	TC02 to TC06, TC19
S06	Web UI and REST API	React app and FastAPI under /api/v1	TC31
S07	New quiz starts as Draft	create_quiz in quizzes.py	TC01
S08	Valid, invalid and unauthorized tests	tests/test_api.py (30 tests)	Whole suite

10.2 Functional requirements

FR	Short description	Tests
FR01-FR02	Create a quiz; validate title and description	TC01 to TC03
FR03	Add, edit, remove questions on a Draft quiz	TC07, TC08, TC11
FR04-FR05	Edit details and delete only while Draft	TC11, TC12
FR06	Publish needs at least 1 valid question	TC09, TC10
FR07	Student sees Published quizzes with attempted flag	TC13, TC14
FR08	Start an attempt without correct flags; one attempt only	TC15, TC16
FR09-FR10	Submit answers; server computes score in one transaction	TC18, TC19, TC28
FR11	A submitted attempt cannot be changed	TC20
FR12	Student result and per-question review	TC22, TC23
FR13	Teacher results and summary figures	TC24
FR14	Closing stops new attempts, keeps results	TC17, TC25

10.3 Final review

- All S requirements (S01 to S08) and the adopted P rules are implemented.
- Business rules are enforced on the server and in the database, not only in the browser.
- 30 of 30 automated tests pass; TC30 (accessibility and narrow screen) remains a manual check.
- Features the SDD excludes (timers, retakes, registration, reports, notifications) were not built.
- Known gap: there are no automated browser (UI) tests; the screens were checked manually.
- Decisions where the SDD was silent: the database uses port 5433; GET /attempts accepts a status filter; an In Progress attempt can be reloaded without correct answers; the frontend folder is named sdd.

11. How to run the project

Three things must be running: the database, the API and the React app. Open three terminals.

What	Folder	Command
Database	react project 6	docker compose up -d db
Backend (first time)	react project 6\backend	.\venv\Scripts\Activate.ps1 then alembic upgrade head then python seed.py
API	react project 6\backend	uvicorn app.main:app --reload --port 8000
React app	react project 6\sdd	npm install (first time) then npm run dev
Tests	react project 6\backend	python -m pytest -v

Open http://localhost:5173. The API documentation is at http://localhost:8000/docs.

Role	Email	Password
Teacher	teacher@example.com	Teacher@123
Student	student1@example.com	Student@123
Student	student2@example.com	Student@123

These are test accounts for a local demo only. Real passwords and the JWT secret in backend/.env must never be shared or committed.

11.1 Ideas for later versions

- Quiz timer with automatic submit when time is over.
- Retakes with best-score or latest-score rules.
- More question types and a question bank.
- Registration and password reset screens, or single sign-on.
- Browser tests for the Teacher and Student journeys, and charts for results.

Appendix A. Project structure and data

A.1 Folder structure

```
react project 6/
  docker-compose.yml        PostgreSQL 16 (port 5433)
  TEST_RESULTS.md           written by the test run
  docs/                     the SDD PDF
  backend/
    .env                    DATABASE_URL, JWT_SECRET, JWT_EXPIRE_MINUTES
    seed.py                 creates the test users
    pytest.ini
    alembic/                migrations
    app/
      main.py               app, CORS, routers
      core/                 config.py database.py errors.py security.py deps.py
      models/models.py      six tables
      schemas/              auth.py quiz.py question.py attempt.py
      routers/              auth.py quizzes.py questions.py attempts.py
      services/ repositories/   (layers reserved by the SDD structure)
    tests/                  conftest.py test_api.py
  sdd/                      React app (Vite)
    .env                    VITE_API_URL
    src/
      main.jsx App.jsx index.css
      api/client.js
      auth/                 AuthContext.jsx ProtectedRoute.jsx
      components/           Layout.jsx QuestionForm.jsx
      pages/                Login MyQuizzes NewQuiz QuizEditor QuizResults
                            AvailableQuizzes TakeQuiz ResultDetail MyResults
```

A.2 Database tables

Table	Columns
users	id, name, email (unique), password_hash, role, is_active, created_at
quizzes	id, teacher_id, title, description, status, created_at, updated_at, published_at, closed_at
questions	id, quiz_id, text, position, created_at
options	id, question_id, text, is_correct, position
attempts	id, quiz_id, student_id, status, score, total, percentage, started_at, submitted_at; unique (quiz_id, student_id)
attempt_answers	id, attempt_id, question_id, option_id (nullable); unique (attempt_id, question_id)

A.3 API endpoints (18)

Method and path	Role
GET /health	Public
POST /auth/login	Public
GET /auth/me	Any logged-in user
POST /quizzes	Teacher
GET /quizzes	Teacher (own) and Student (Published)
GET /quizzes/{id}	Teacher owner, Student (Published summary)
PATCH /quizzes/{id}	Teacher owner, Draft only
DELETE /quizzes/{id}	Teacher owner, Draft only
POST /quizzes/{id}/publish	Teacher owner

Method and path	Role
POST /quizzes/{id}/close	Teacher owner
POST /quizzes/{id}/questions	Teacher owner, Draft only
PUT /quizzes/{id}/questions/{qid}	Teacher owner, Draft only
DELETE /quizzes/{id}/questions/{qid}	Teacher owner, Draft only
POST /quizzes/{id}/attempts	Student
POST /attempts/{id}/submit	Student (own attempt)
GET /attempts	Student
GET /attempts/{id}	Student (own) and Teacher (own quiz)
GET /quizzes/{id}/results	Teacher owner

All paths start with /api/v1. Errors use one format: {error: {code, message, fields, request_id}}.